

# ScholarMaster: A Layered Edge-Native Architecture for Real-Time Context-Aware Intelligent Systems

Polisetty Narendra\*, Suresh Kumar Sreedharan<sup>†</sup>, Vijaya Raju Motru<sup>‡</sup>, and Praveena Mallampalli<sup>§</sup>

Swarnandhra College of Engineering & Technology (Autonomous)

Narsapur, Andhra Pradesh, India

\*ORCID: 0009-0002-6355-138X, <sup>†</sup>ORCID: 0000-0001-9912-5927

<sup>‡</sup>vijayaraju.m@gmail.com, <sup>§</sup>praveena.iduri@gmail.com

**Abstract**—Modern intelligent systems are increasingly required to operate under stringent real-time, privacy, and resource constraints, particularly at the edge. While recent advances in individual subsystems such as perception, inference, and control have contributed to improved performance at the component level, the lack of architectural principles has typically resulted in tightly coupled monolithic systems that are challenging to evaluate in isolation.

A system’s degree of decomposition is a fundamental property in architectural synthesis - ScholarMaster takes this to be a given. By synthesizing results of a multi-paper research agenda, this paper proposes an edge-native framework that isolates the context-aware pipeline into focused domains: sensing, geometric abstraction, probabilistic interpretation, logical evaluation, and control.

By ensuring strict abstraction admissibility between these stages, the architecture greatly limits cross-layer leakage. This, in turn, allows for an overall design that is modular by construction. By building on previous results in independently verifiable sub-systems, we proceed to demonstrate the benefits of this deterministic composition over more monolithic approaches, particularly in the face of interpretability, fault confinement, and auditability concerns.

**Index Terms**—System Architecture, Edge-Native, Boundary Invariants, Deterministic Composition, Abstraction Admissibility, Fault Containment.

## I. Introduction

The central thesis of this paper is that architectural boundary invariants can enable decomposition to be a verifiable systems property for edge-native intelligent systems. This manuscript is exclusively a systems-architecture paper, focusing on investigating edge-native structural composition, and not a general-purpose statement about intelligent-campus design or policy.

### A. Operational Fragility in End-to-End Intelligent Systems

Deploying intelligent systems in institutional cyber-physical environments [10], [12], [19] presents a broad range of interlocking operational constraints [6]. Monolithic edge pipelines are often insufficient, as tightly coupled representations hide interlocking dependencies, upgrades destabilize unrelated subsystems, audits fail to reconstruct lineage, and implicit assumptions dominate interactions across layers. There are no clean solutions

because the challenges are deeply entangled with the architecture.

Over the course of developing the ScholarMaster subsystem ecosystem, these architectural failure patterns became apparent [7], [16], [21]:

- 1) **End-to-End Monoliths:** Optimization pressures favor direct, opaque coupling. While performant in static benchmarks, these pipelines proved brittle under incremental evolution [11].
- 2) **Cloud-Modular Dependencies:** Offloading reasoning introduces temporal jitter and privacy exposure [13], [17].
- 3) **Loose Event-Driven Buses:** Exhibit “abstraction drift”—subsystems operating on conflicting temporal views accumulate synchronization inconsistencies.

### B. Subsystem Scope and Canonical Separation

Within the ScholarMaster architecture, this particular subsystem is responsible for specifying architectural strata, for establishing abstraction admissibility criteria, and for restricting inter-layer interactions. This subsystem does NOT realize probabilistic engagement interpretation (Paper 2), perform geometric sensing (Paper 3), execute runtime verification proofs (Paper 18), coordinate orchestration policy (Paper 9), or manage cryptographic provenance (Paper 8).

The ScholarMaster architecture has been defined through the insight that decomposition must be made obligatory. The present paper attempts to capture the architectural knowledge generated within the research program by synthesizing separately established approaches and formalisms into a reference architecture.

## II. Related Architectural Paradigms

The architectural decisions made behind the design of ScholarMaster are best explained through the shortcomings of other existing intelligent system frameworks. There are three main approaches that have informed the building principles of present-day edge-intelligent systems, and each highlights a unique type of decomposition inadequacy in institutional cyber-physical frameworks.

End-to-end monolithic pipelines [14] are widespread because of their Simplicity of implementation and throughput efficiency. The main drawback of such pipelines is the entanglement of representations of various stages in a single one, due to which it becomes difficult to determine which stage is responsible for what. Thus, if such a system makes a wrong decision, a researcher may have to examine all stages to identify the faulty one.

Cloud-native microservice architectures [10], [18] address this coupling through network-level decomposition, but introduce a different failure mode. In edge environments, serialization and network transit overhead produces temporal jitter that undermines the determinism required for cyber-physical synchronization [20]. More subtly, loosely constrained service interfaces allow hidden semantic dependencies to emerge across layers, recreating at the API level the same entanglement that monoliths exhibit at the memory level.

Event driven distributed pipelines [3] address both scalability and modularity concerns to some extent, but introduce the notion of synchronization drift: different elements operating on inconsistent temporal snapshots of the system state diverge in their logical state; unless explicitly mediated, this leads to violations of synchronization contracts under load or degraded operating conditions.

A commonality of these approaches in this regard is that in none of them is considered a decomposition boundary as an architecturally mandated invariant. Rather, boundaries are viewed as implementation conveniences that can be violated with little or no penalty. In practice, this perspective has proven to have significant negative consequences for large-scale software systems, which is discussed in detail in the next subsection. In particular, during independent module verification for our subsystem studies within the ScholarMaster program, such violations have been consistently found to occur under optimization pressure. By contrast, the architecture described emphasizes on the importance of boundaries as a first-class concern (Table I), and places greater emphasis on ensuring deterministic composition of subsystems, their orthogonal abstraction, and auditability of such composition.

TABLE I  
Architectural Paradigm Comparison

| Paradigm        | Primary Weakness      | ScholarMaster Alternative           |
|-----------------|-----------------------|-------------------------------------|
| Monolithic AI   | Opaque coupling       | Boundary invariants                 |
| Microservice AI | Weak determinism      | Edge memory confinement             |
| Event-Driven    | Synchronization drift | Upward abstraction<br>admissibility |

### III. Boundary Invariants and Formalisms

Several operational constraints were identified while performing layered decomposition of the ScholarMaster

subsystems. Resolving these constraints resulted in specification of a set of boundary invariants that impose abstraction admissibility requirements on the decomposed subsystems. Decomposition is imposed as an operational constraint, not a design preference. The boundaries are enforced structurally, not just by organization, since it is important to distinguish interface contracts, abstraction admissibility, ownership isolation, and prevention of semantic leakage.

#### A. Abstraction Admissibility

Modularity is maintained by constraining each layer  $L_i$  to a bounded transformation:

$$L_i : X_i \rightarrow X_{i+1} \quad (1)$$

where  $X_i$  represents the minimal abstraction admissible for upward propagation. Higher layers consume strictly immutable abstractions. A sensing layer may output geometric coordinates, but it is structurally incapable of emitting a logical predicate. This constraint prevents higher-level logic from becoming implicitly dependent on lower-level implementation details [1], [8].

#### B. Edge-Native Memory Confinement

Computation takes place in local, volatile, hardware. By removing the dependency on the cloud, it removes network latency, and ensures that data is stored wherever it is needed. Previous systems established that treating all external links as either compromised or down is necessary to ensure that logic on critical paths remains isolated.

#### C. Deterministic Composition

Determinism emerges from constrained architectural interaction. Synchronous execution achieves predictability by considering outputs from lower layers in their computation as deterministic inputs. Skipping layers creates ambiguity and is explicitly disallowed. Traceability comes from restricted uplinking. The resulting behavior is thus formally traceable within the constraints, and any emergent properties are predictable and not subject to the ambiguity of end-to-end deep learning architectures where representations are not formally defined.

### IV. The ScholarMaster Reference Model: A 4-Stratum Architecture

The architecture is organized into four levels in which the system is separated into different areas of functionality and access. This division makes sure that the raw physical observations are not intertwined with the decision-making processes.

- Physics Stratum (L1-L2): Manages raw sensing and geometric abstraction. Subsystems in this stratum extract objective coordinates while remaining agnostic to institutional identity.
- Logic Stratum (L3-L4): Converts geometric abstractions into probabilistic states and logical predicates.

This stratum handles uncertainty propagation and enforces the Boolean evaluation rules governing system behavior.

- Verification Stratum (L5-L6): Ensures the system operates within its temporal and formal bounds, enforcing sub-millisecond scheduling and architectural isolation.
- Governance Stratum (L7-L8): Handles auditing, privacy preservation, and institutional reporting, managing the SHA-256 chains that allow auditing without exposing raw data.

Figure 1 visualizes the required abstraction contracts and illustrates the architectural hazards of bypassing them.

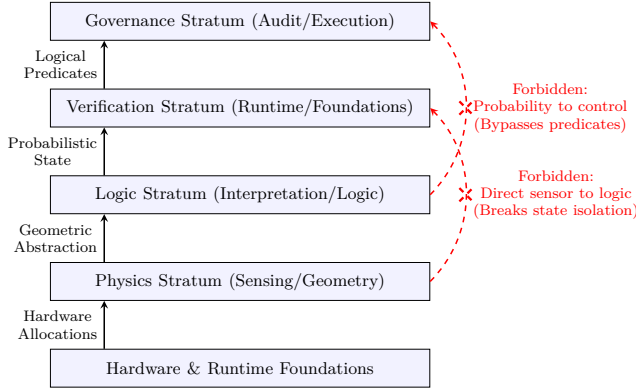


Fig. 1. The ScholarMaster Layered Architecture. Allowed interfaces mandate progressive abstraction (left). Lateral or skip-level communication is explicitly forbidden to prevent cross-layer leakage (right).

#### A. Constraint-First Architectural Synthesis (CFAS)

The ScholarMaster architecture was created using the innovative method called Constraint-First Architectural Synthesis (CFAS). This method focuses on the reverse engineering of AI-system building principles. The boundaries are defined before features are created and installed within the system. The subsystem’s acceptability is prioritized with regard to the overall design’s survivability as opposed to its immediate throughput. In other words, instead of defining the set of features that the system will be designed around, and later installing the privacy and real-time guarantees, the features were defined according to the constraints. The reason for this approach was determined during the prototyping phase, as most of the feature-first designs resulted in the necessity to implement costly privacy retrofits.

#### B. Schema-Constrained Semantic Firewalls

Schema constraints dictate what abstractions are permissible, but provide no security guarantees. The architecture creates an “Irreversibility Boundary” between layers via schema-constrained semantic firewalls. By enforcing strict typing via serialization (e.g. Protobuf), representation leakage is mechanically inhibited while abstraction

narrowing reduces downstream semantic dependencies, thus making it impossible to pass a raw image tensor or absolute coordinate system across layer boundaries. For instance, the interface between the Sensing and Reasoning layer only permits categorical IDs and logical spatial regions, preventing identity leakage at the feature level.

#### V. Cross-Layer Consistency and the Research Ecosystem

The ScholarMaster ecosystem decomposes architectural responsibility into independently verifiable domains spanning sensing, reasoning, runtime enforcement, and governance. This decomposition evolved organically as subsystem complexity grew: what began as a monolithic prototype progressively fractured into distinct ownership domains as operational tensions between layers became apparent. Each subsystem now evolves under independently auditable constraints while preserving global operational consistency. Individual subsystem formalizations are documented across the broader ScholarMaster research program (Table II).

TABLE II  
Subsystem Ownership Mapping

| Subsystem Domain             | Primary Formalization Source |
|------------------------------|------------------------------|
| Vision Geometry              | SM-P3                        |
| Acoustic Sensing             | SM-P6                        |
| Probabilistic Interpretation | SM-P2                        |
| Logical Evaluation           | SM-P4                        |
| Runtime Scheduling           | SM-P20                       |
| Formal Foundations           | SM-P21                       |
| Privacy Governance           | SM-P8                        |
| Identity Retrieval           | SM-P7                        |
| Reference Architecture       | SM-P18                       |
| Threat Modeling              | SM-P19                       |
| Control Dispatch             | SM-P9                        |
| State Management             | SM-P11                       |

#### VI. Example Operational Trace: Attendance Validation

To ground the preceding formalism in operational reality, consider a concrete institutional attendance validation event. This trace illustrates the layered traceability that the architecture enforces across a single decision lifecycle.

The process starts with the Sensing Layer that extracts the raw geometric occupancy abstractions (bounding boxes, skeletal points) from the visual and acoustic streams. Note that the Sensing Layer has no awareness of “who” is present or “why” - it provides stateless physical coordinates.

These abstractions are funneled into the Probabilistic Interpretation Layer, where they are anchored to temporal priors in order to define bounded confidence states. This layer defines the probability of existence and defines away noise in favor of persistence.

The resulting probability state is then fed into the next component, the Logical Evaluation Layer, which evaluates institution-specific predicates (e.g., time thresholds or allowed schedules). The Control Layer only proceeds to

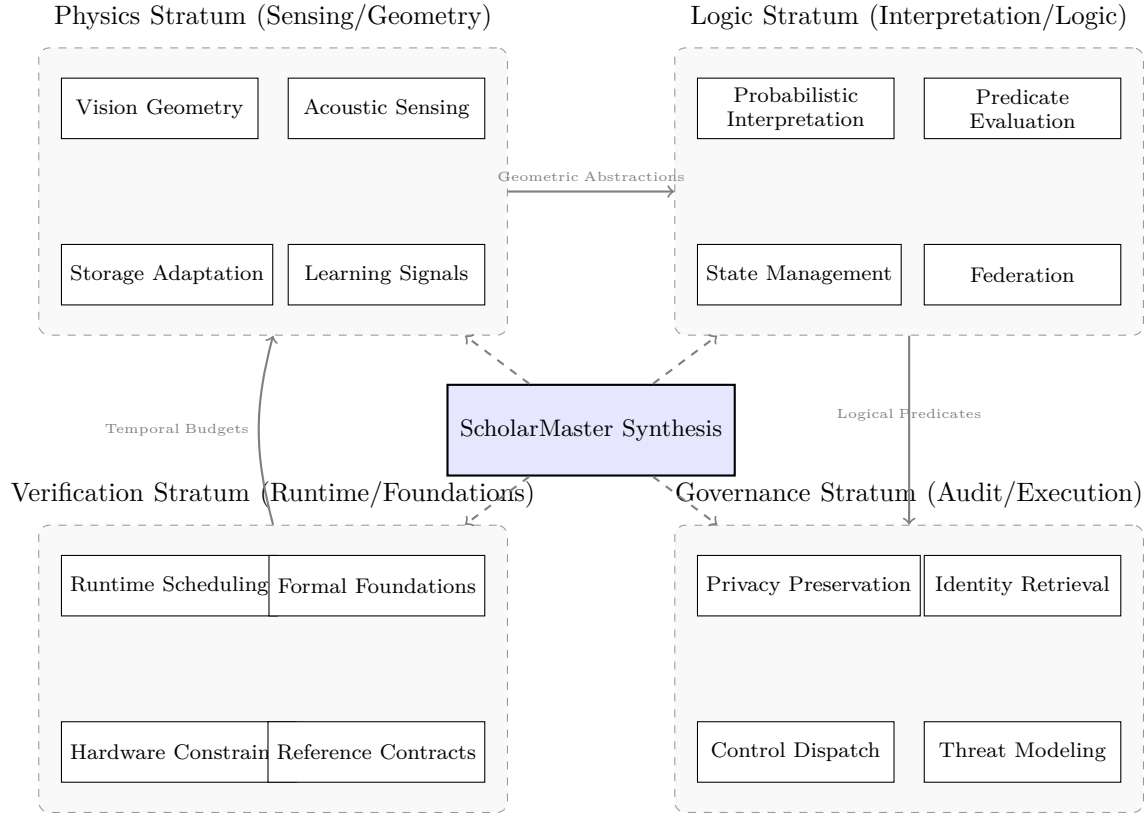


Fig. 2. The ScholarMaster Research Ecosystem Map. The architecture decouples system responsibility into four independently verifiable layers (Physics, Logic, Verification, Governance). Here, we see how the system enforces a set of boundaries that allows the absence of uncontrolled cross-layer interactions.

perform an operation (e.g., update a ledger or allow access) if the predicate is satisfied deterministically.

At no point do we ever allow the control layer to observe raw sensor tensors, or the sensing layer to understand institutional policy semantics. If an action is challenged, an auditor can reconstitute the precise logical predicate and associated probability that led to it, without having to watch a single frame of video.

## VII. Architectural Hazards of Cross-Layer Leakage

Across the subsystem studies, a consistent pattern emerged: the most operationally significant failures in edge-intelligent systems rarely originated from algorithmic weakness within a single layer. Rather, they traced back to uncontrolled information flow across layer boundaries [2].

When a vision layer accumulates temporal tracking state that belongs to the interpretation layer or when control loops read raw sensor confidence scores, without predicate-based mediation, the resulting system is no longer accountable; Table III specifies the architecture’s formal prohibitions.

By enforcing this functional ownership, an auditing body can inspect the Logical Evaluation Layer in isolation,

TABLE III  
Examples of Forbidden Cross-Layer Behaviors

| Architectural Violation                      | Why it is Strictly Forbidden                                               |
|----------------------------------------------|----------------------------------------------------------------------------|
| Vision layer making logical access decisions | Violates separation of concerns; locks logic inside perception weights.    |
| Runtime layer modifying inference priors     | Breaks deterministic composition; causes hardware load to alter reasoning. |
| Logic layer caching raw sensor streams       | Violates data minimization; creates severe privacy liabilities.            |

thus ensuring that it does not rely on any implicit state injection from perception or runtime modules.

## VIII. Operational and System-Level Properties

The operational properties described in this section were not designed prescriptively. They emerged as consistent observations during independent subsystem validation and integration testing across the research program.

In institutional settings, cross-layer leakage rarely causes outright system failures. Instead, it leads to operational ambiguities, as organizations grapple with the realization that a behavioral anomaly might have

emerged from sensing or probabilistic inference, logical policy inconsistency, or runtime scheduling imprecision. ScholarMaster addresses such ambiguities by design - by creating an architecture that makes it impossible for such anomalies to manifest without a clear owner function, thereby constraining their source to a single, verifiable cause.

This concern carries over to architectural debt, which in intelligent systems takes a different form compared to traditional software. In tightly coupled AI pipelines, implicit couplings between various stages can appear in the form of hidden dependencies between intermediate representations, temporal assumptions, memory layouts, and more. These can be discovered too late, when an upgraded sub-system touches upon these points, potentially requiring a significant validation effort of the entire pipeline. The boundaries imposed by the invariants of ScholarMaster on the interactions between its layers, however, prevent such architectural debt from appearing: if the coupling between two sub-systems is performed via admissible abstractions, any local changes to one of them can only affect its own layer, not the other.

#### A. Failure Containment and Propagation

In monolithic systems, local perception failures often propagate catastrophically to higher levels since further processing steps operate directly on unstable representations. For example, an intermittent drop in confidence during low-light tracking could lead to thruster control loops executing unsafe, rapidly alternating control decisions with no inherent failsafes.

ScholarMaster constrains this propagation through layered failure containment (Figure 3). If a sensing layer is degraded, higher layers see only a bounded abstraction degradation instead of outright instability. The probability layer adds an additional constraint smoothing out the mathematical space, while the logic layer can revert to a simple Boolean default to bound the operational impact of localized sensing failures [8].

#### B. Operational Auditability

Auditability in ScholarMaster is done by construction and not by logging after the fact since each stage produces/consumes only bounded abstract representations. This allows for reconstructing the decision making process of the system at any point in time with full knowledge of the relevant constraints without having to reason about lower level sensor details or higher level memory structures. The auditor can reason about the exact geometric abstraction that produced a particular probability and the exact probability threshold that triggered a logical predicate.

Table IV summarizes how these emergent properties compare against traditional monolithic designs across key operational dimensions.

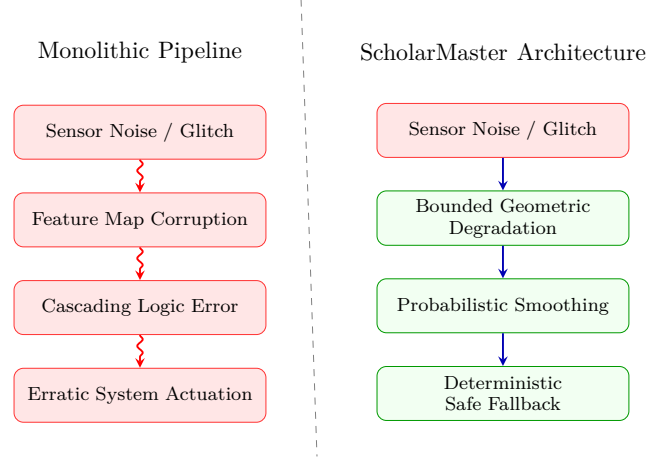


Fig. 3. Failure Propagation Comparison. While monolithic systems allow sensor glitches to corrupt global logic, ScholarMaster’s boundary invariants force failures to undergo structural containment and safe probabilistic degradation.

TABLE IV  
System Property Comparison

| Property                | Monolith  | ScholarMaster |
|-------------------------|-----------|---------------|
| Hidden coupling         | High      | Low           |
| Upgrade fragility       | High      | Low           |
| Audit reconstruction    | Weak      | Strong        |
| Failure isolation       | Weak      | Strong        |
| State ownership clarity | Ambiguous | Explicit      |

#### IX. Lifecycle Stability and Upgrade Semantics

A persistent operational problem observed during longitudinal subsystem development is upgrade fragility. In tightly coupled architectures, updating a perception model frequently invalidates downstream logic assumptions, forcing expensive end-to-end revalidation. Hardware ages, models decay, and systems without isolation eventually become entangled with their oldest component.

ScholarMaster addresses this problem in the form of admissible-state contracts. It was worked out during the evolution of subsystems, which allows for some internal implementation changes so long as the exported abstraction boundaries are maintained. Individual layers can be modified independently of one another, thus leaving the overall system intact.

For example, during the research program, the geometric sensing sub-system was replaced by an entirely different architecture (CNN-based detector replaced with transformer-based representation model) while keeping the same probabilistic interpretation and logical evaluation layers intact. As long as the exported geometric abstraction adhered to the interface contract, the higher level reasoning modules were essentially oblivious to the lower level changes. Such properties are particularly valuable in practice as they significantly reduce the long-term maintenance overhead and mitigate risks of silent

behavioral regressions during iterative improvements.

## X. Institutional Deployment and Multi-Tenant Realities

Institutional intelligent systems rarely operate in isolation. Universities, hospitals, and industrial facilities often have multiple operational policies that must be enforced concurrently on shared hardware. Experience with the implementation of individual subsystems has shown that clear ownership of policy fragments is critical for manageability.

Because the operational logic is decoupled from the sensing infrastructure, policy layers can be upgraded independently of each other, without requiring re-certification of lower levels. This is particularly important in a regulated environment, where the evolution of the governance logic may be faster than the approval cycles for the underlying sensing hardware. Moreover, multi-tenancy is possible, because the logical evaluation layer is separated from sensing and execution, and different policy domains can be executed concurrently, without exposing lower-level sensor data to other organizations.

This decomposition also allows selective recertification. For instance, a change in institutional logic (e. g., attendance policy) does not require re-certifying sensing hardware or re-evaluating probabilistic interpretations. This dramatically lowers the governance overhead, enabling business logic to evolve faster than the supporting hardware.

## XI. Certification and Regulatory Implications

Edge intelligent systems tend to function in environments where regulations require explanations, audit trails, and a limited behavior set. Certification is often challenging for such systems if perception weights cause direct business logic effects, as is the case with many monolithic deep learning systems, since they are typically not directly tied to business logic.

ScholarMaster tackles this by slicing up the responsibility surface into independently verifiable layers so that any given component can be certified for logical or control behavior without diving into implementation details below. The design reflects institutional governance frameworks like GDPR [15], where data minimization and deterministic reconstruction of operational logic are mandated features, not optional extensions.

### A. Architectural Scope and Economic Tradeoffs

ScholarMaster focuses on application domains where interpretability, lifecycle auditing, predictable composition, and bounded execution profiles matter more than throughput optimization. Thus, it has a higher latency floor than specialized fused pipelines that prioritize performance isolation.

A practical intelligent system rarely stays the same throughout its lifespan. Hardware diverges, versions skew between generations of a subsystem, performance degrades on older hardware, serialized processing introduces

overhead, and institutional deployment diversity leads to heterogeneity. The architecture given is designed around an expectation of ongoing evolution rather than uniform upgrades of an entire stack at once, and thus emphasizes subsystem survivability in the face of iterative replacement.

Each abstraction boundary in ScholarMaster incurs costs such as serialization overhead, interface verification, and reduced ownership. Systems with tighter coupling often seem more cost-effective during implementation, as direct memory access reduces immediate engineering costs [11]. However, these costs are often deferred to later stages in the development cycle.

### B. Failed Design Alternatives

Early architectural prototypes explored shared-memory fusion between the sensing and logic layers, on the premise that raw memory pointers would eliminate serialization copy overhead. The approach reduced end-to-end latency by several milliseconds, but produced hidden coupling: the logic layer became implicitly dependent on the memory layout of the vision pipeline. When the vision model was quantized for deployment, downstream logic failed without any interface contract violation being reported. This experience led the architecture to reject shared-memory fusion, accepting serialization overhead as the structural cost of independent verifiability.

### C. Hard Architectural Limitations

This framework is not a universal solution. The architecture intentionally prioritizes modularity and auditability over peak throughput optimization. We explicitly acknowledge the architectural tradeoff: the framework accepts higher latency floors relative to fused pipelines, significant interface-maintenance overhead, increased engineering discipline requirements, and deployment complexity in ultra-low-latency domains. ScholarMaster is explicitly scoped to edge-native domains where interpretability, deterministic containment, and lifecycle maintainability are operationally more valuable than marginal throughput gains.

## XII. Conclusion

This layered edge-native architecture study demonstrates that scaling intelligent systems at the edge benefits from sustained architectural discipline alongside algorithmic optimization. Formalizing boundary invariants and enforcing abstraction admissibility across the processing pipeline substantially reduces the brittleness that accumulates in monolithic designs over operational lifetimes.

This comes at a cost. Constraining perception, interpretation, logic, and control to interact exclusively through narrow, typed interfaces introduces serialization overhead and engineering friction that fused pipelines avoid. Across the independently validated subsystem studies comprising this research program, however, that friction consistently

contributed to failure containment, audit reconstructibility, and independent subsystem evolution—properties that proved more operationally valuable than the marginal latency savings of tighter coupling.

This paper collates systems-engineering understanding distilled from a larger body of work represented in the papers that follow. The resulting design philosophy cannot be considered universal; rather, it is appropriate to the class of systems comprised of edge-native institutions for which interpretability, deterministic composition, abstraction admission, and evolution resistance are primary concerns. To the extent that these criteria are relevant to the systems under consideration, the evidence presented supports the validity of their decomposition as a system property that can be enforced with reasonable cost and risk.

## References

- [1] D. L. Parnas, "On the criteria to be used in decomposing systems into modules," *Communications of the ACM*, vol. 15, no. 12, pp. 1053-1058, 1972.
- [2] A. Avizienis, J. Laprie, B. Randell, and C. Landwehr, "Basic concepts and taxonomy of dependable and secure computing," *IEEE Transactions on Dependable and Secure Computing*, vol. 1, no. 1, pp. 11-33, 2004.
- [3] P. Th. Eugster, P. A. Felber, R. Guerraoui, and A.-M. Kermarrec, "The many faces of publish/subscribe," *ACM Computing Surveys*, vol. 35, no. 2, pp. 114-131, 2003.
- [4] M. Quigley et al., "ROS: an open-source Robot Operating System," in *ICRA workshop on open source software*, 2009.
- [5] N. G. Leveson, *Engineering a Safer World: Systems Thinking Applied to Safety*. MIT Press, 2011.
- [6] H. Kopetz, *Real-Time Systems: Design Principles for Distributed Embedded Applications*. Springer Science & Business Media, 2011.
- [7] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog computing and its role in the internet of things," in *Proceedings of the first edition of the MCC workshop on Mobile cloud computing*, 2012, pp. 13-16.
- [8] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 3rd ed. Addison-Wesley Professional, 2012.
- [9] J. A. Stankovic, "Research directions for the internet of things," *IEEE Internet of Things Journal*, vol. 1, no. 1, pp. 3-9, 2014.
- [10] S. Newman, *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, Inc., 2015.
- [11] D. Sculley et al., "Hidden technical debt in machine learning systems," in *Advances in neural information processing systems*, vol. 28, 2015.
- [12] E. A. Lee, "The Past, Present and Future of Cyber-Physical Systems: A Focus on Models," *Sensors*, vol. 15, no. 3, pp. 4837-4869, 2015.
- [13] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge Computing: Vision and Challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637-646, 2016.
- [14] M. Bojarski et al., "End to end learning for self-driving cars," *arXiv preprint arXiv:1604.07316*, 2016.
- [15] P. Voigt and A. von dem Bussche, *The EU General Data Protection Regulation (GDPR): A Practical Guide*. Springer International Publishing, 2017.
- [16] M. Satyanarayanan, "The Emergence of Edge Computing," *Computer*, vol. 50, no. 1, pp. 30-39, 2017.
- [17] R. Roman, J. Lopez, and M. Mambo, "Mobile edge computing, fog et al.: A survey and analysis of security threats and challenges," *Future Generation Computer Systems*, vol. 78, pp. 680-698, 2018.
- [18] P. Jamshidi et al., "Microservices: The Journey So Far and Challenges Ahead," *IEEE Software*, vol. 35, no. 3, pp. 24-35, 2018.
- [19] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge Intelligence: Paving the Last Mile of Artificial Intelligence With Edge Computing," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1738-1762, 2019.
- [20] Y. Gan et al., "An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems," in *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019, pp. 3-18.
- [21] X. Wang et al., "Convergence of Edge Computing and Deep Learning: A Comprehensive Survey," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 2, pp. 869-904, 2020.
- [22] S. Suresh Kumar, "Perception Integrity Foundations: Evidential Uncertainty and Calibrated Disagreement in Edge Vision," *ScholarMaster Research Series*, Paper 22, 2026.
- [23] S. Suresh Kumar, "ScholarMaster Integration Architecture and Downstream Error Propagation Analysis," *ScholarMaster Research Series*, Paper 25, 2026.
- [24] S. Suresh Kumar, "Perception integrity foundations: Evidential uncertainty, disagreement dynamics, and blur bounds in edge vision," *ScholarMaster Technical Report Series*, Paper 22, 2026.
- [25] S. Suresh Kumar, "ScholarMaster macro integration architecture and downstream error propagation analysis," *ScholarMaster Technical Report Series*, Paper 25, 2026.